

# Data Collection Layer - Crypto Sentiment Analytics

## Big Data Analytics Final Project

**Team:** Emre Akyol, Harmanpreet Chauhan, Mohamed Nasr

---

This notebook implements the Data Source Layer, collecting real-time cryptocurrency data from CoinGecko API, Alternative.me (Fear & Greed Index), and CryptoPanic (news sentiment).

```
In [1]: import requests
import json
import pandas as pd
import numpy as np
from datetime import datetime, timedelta
import time
import os
import warnings
warnings.filterwarnings('ignore')

import matplotlib.pyplot as plt
import seaborn as sns
sns.set_style('darkgrid')

os.makedirs('./resources/data', exist_ok=True)
print(f"Setup complete - {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}")
```

Setup complete - 2026-01-04 22:03:01

## CoinGecko API - Cryptocurrency Prices

```
In [2]: CRYPTO_IDS = {'BTC': 'bitcoin', 'ETH': 'ethereum', 'XRP': 'ripple', 'SOL': 'solana'}

def fetch_crypto_details(coin_id, symbol):
    try:
        url = f"https://api.coingecko.com/api/v3/coins/{coin_id}"
        params = {'localization': 'false', 'tickers': 'false', 'community_data': 'false'}
        response = requests.get(url, params=params, timeout=15)

        if response.status_code == 200:
            data = response.json()
            md = data.get('market_data', {})
            sparkline = md.get('sparkline_7d', {}).get('price', [])
            now = datetime.now()
            timestamps = [(now - timedelta(hours=len(sparkline)-i-1)).isoformat() for i in range(len(sparkline))]

            return {
                'symbol': symbol,
                'name': data.get('name', symbol),
```

```

        'price': float(md.get('current_price', {}).get('usd', 0) or 0),
        'marketCap': float(md.get('market_cap', {}).get('usd', 0) or 0),
        'volume24h': float(md.get('total_volume', {}).get('usd', 0) or 0),
        'change24h': float(md.get('price_change_percentage_24h', 0) or 0),
        'change7d': float(md.get('price_change_percentage_7d', 0) or 0),
        'volatility': float(abs(md.get('price_change_percentage_24h', 0) or 0)),
        'sparkline': [float(p) for p in sparkline],
        'sparkline_timestamps': timestamps,
        'high24h': float(md.get('high_24h', {}).get('usd', 0) or 0),
        'low24h': float(md.get('low_24h', {}).get('usd', 0) or 0)
    }
    return None
except Exception as e:
    print(f"Error: {e}")
    return None

```

```

In [3]: print("Fetching cryptocurrency data...\n")
        crypto_data_list = []

        for i, (symbol, coin_id) in enumerate(CRYPTO_IDS.items(), 1):
            print(f"[{i}/6] {symbol}...", end=' ')
            data = fetch_crypto_details(coin_id, symbol)
            if data:
                crypto_data_list.append(data)
                print(f"${data['price']:,}.2f" if data['price'] >= 1 else f"${data['price']}")
            if i < len(CRYPTO_IDS): time.sleep(4)

        print(f"\nFetched {len(crypto_data_list)} cryptocurrencies")
        df_crypto = pd.DataFrame(crypto_data_list)
        display(df_crypto[['symbol', 'name', 'price', 'change24h', 'volatility']])

```

Fetching cryptocurrency data...

```

[1/6] BTC... $91,245.00
[2/6] ETH... $3,138.72
[3/6] XRP... $2.09
[4/6] SOL... $133.91
[5/6] DOGE... $0.149850
[6/6] ADA...

```

Fetched 5 cryptocurrencies

|   | symbol | name     | price       | change24h | volatility |
|---|--------|----------|-------------|-----------|------------|
| 0 | BTC    | Bitcoin  | 91245.00000 | 1.05108   | 0.010511   |
| 1 | ETH    | Ethereum | 3138.72000  | 0.81855   | 0.008186   |
| 2 | XRP    | XRP      | 2.09000     | 4.19362   | 0.041936   |
| 3 | SOL    | Solana   | 133.91000   | 1.54099   | 0.015410   |
| 4 | DOGE   | Dogecoin | 0.14985     | 5.35820   | 0.053582   |

## Alternative.me - Fear & Greed Index

```
In [4]: def fetch_fear_greed():
    try:
        response = requests.get('https://api.alternative.me/fng/?limit=30', timeout
        if response.status_code == 200:
            data = response.json()['data']
            return {
                'current': int(data[0]['value']),
                'classification': data[0]['value_classification'],
                'history': [{ 'value': int(d['value']), 'date': d['timestamp']} for
            }
    except: pass
    return {'current': 50, 'classification': 'Neutral', 'history': []}

fear_greed = fetch_fear_greed()
print(f"Fear & Greed Index: {fear_greed['current']} ({fear_greed['classification']})")
```

Fear & Greed Index: 25 (Extreme Fear)

## CryptoPanic - News Sentiment

```
In [5]: def fetch_news():
    try:
        url = "https://cryptopanic.com/api/free/v1/posts/"
        response = requests.get(url, params={'auth_token': 'f72c9f31848a598c6386827
        if response.status_code == 200:
            posts = response.json().get('results', [])[:30]
            return [{
                'title': p.get('title', ''),
                'source': p.get('source', {}).get('title', 'Unknown'),
                'url': p.get('url', ''),
                'votes': {'positive': p.get('votes', {}).get('positive', 0), 'negat
                'sentiment': {'combined': round((p.get('votes', {}).get('positive',
                'currencies': [c.get('code', '') for c in p.get('currencies', [])]
            } for p in posts]
    except: pass
    return []

news_items = fetch_news()
print(f"Fetched {len(news_items)} news headlines")
for n in news_items[:3]: print(f" - {n['title'][:60]}...")
```

Fetched 0 news headlines

## Save Data

```
In [6]: fg_value = fear_greed['current']
social_sentiment = (fg_value - 50) / 50

for crypto in crypto_data_list:
    crypto['socialSentiment'] = float(round(social_sentiment + (crypto['change24h']
    crypto['buzzVolume'] = int(crypto['volume24h'] / 1e6)

btc = next((c for c in crypto_data_list if c['symbol'] == 'BTC'), None)
total_mc = sum(c['marketCap'] for c in crypto_data_list)
```

```

btc_dom = float((btc['marketCap'] / total_mc * 100) if btc and total_mc > 0 else 60)

output_data = {
    'marketOverview': {
        'totalMarketCap': float(total_mc),
        'totalVolume': float(sum(c['volume24h'] for c in crypto_data_list)),
        'btcDominance': round(btc_dom, 1),
        'fearGreedIndex': fg_value,
        'fearGreedClassification': fear_greed['classification'],
        'socialSentiment': float(round(social_sentiment, 2))
    },
    'cryptocurrencies': crypto_data_list,
    'metadata': {'lastUpdate': datetime.now().isoformat(), 'dataSource': 'CoinGecko'}
}

with open('./resources/data/crypto-prices.json', 'w') as f:
    json.dump(output_data, f, indent=2)
print(f"Saved crypto-prices.json ({len(crypto_data_list)} cryptos, ${total_mc/1e12:}

sentiment_data = {
    'sentimentOverview': {'fearGreedIndex': fg_value, 'classification': fear_greed[
    'fearGreedHistory': fear_greed['history'],
    'newsHeadlines': news_items,
    'metadata': {'lastUpdate': datetime.now().isoformat(), 'sources': ['Alternative
}

with open('./resources/data/sentiment-data.json', 'w') as f:
    json.dump(sentiment_data, f, indent=2)
print(f"Saved sentiment-data.json ({len(news_items)} headlines)")

```

Saved crypto-prices.json (5 cryptos, \$2.43T market cap)

Saved sentiment-data.json (0 headlines)

## Summary

Data collected from 3 APIs and saved to JSON files for processing in subsequent notebooks.